Bitácora de desarrollo — GeoCat IA

Proyecto: Sistema multiagente de Catastro y Geodesia
Integrantes: Nikole Flórez y Diana Avendaño
Programa: Ingeniería Catastral y Geodesia
Periodo de desarrollo: noviembre de 2026
Repositorio: https://github.com/NikFlorez3/GeoCat-IA

---

1. Introducción

En esta bitácora presentamos el proceso que seguimos para construir **GeoCat IA**, un proyecto académico en el que exploramos el uso de agentes de inteligencia artificial especializados en Catastro y Geodesia. La propuesta consistió en disponer de un **Coordinador**, encargado de interpretar las consultas, y de dos especialistas que pudieran responder preguntas propias de cada disciplina.

El trabajo se desarrolló en dos frentes relacionados, pero diferentes: por una parte, configuramos agentes dentro de **OpenCode**; por otra, construimos una **implementación funcional en Python** conectada directamente con **Ollama** y los modelos **Qwen**. Durante el proceso instalamos herramientas, organizamos documentos técnicos, ejecutamos pruebas, identificamos errores y publicamos los archivos en GitHub.

La bitácora registra tanto los avances como las dificultades. No buscamos presentar el proyecto como un producto terminado sin limitaciones, sino explicar qué logramos comprobar y qué aspectos quedaron pendientes.

2. Objetivos

Objetivo general

Desarrollar, probar y documentar un sistema multiagente de consulta técnica para Catastro y Geodesia, utilizando modelos de lenguaje ejecutados localmente y herramientas de programación y gestión de versiones.

Objetivos específicos

- Configurar un entorno local con OpenCode, Ollama y modelos Qwen.
- Definir las responsabilidades del Coordinador y de los agentes Catastro y Geodesia.
- Implementar en Python la clasificación de consultas, la recuperación documental y la generación de respuestas.
- Comprobar el comportamiento del sistema mediante preguntas individuales e interdisciplinarias.
- Organizar y publicar el código y la documentación en GitHub.

3. Herramientas empleadas

Herramienta	Uso dentro del proyecto
OpenCode	Configuración y pruebas de agentes nativos
Ollama	Ejecución local de los modelos de lenguaje
Qwen	Generación de respuestas de los especialistas
Python	Desarrollo de la implementación multiagente funcional
PyPDF	Lectura de documentos PDF locales
PowerShell	Instalación, ejecución y verificación de comandos
Git y GitHub	Control de versiones y publicación del repositorio
Hugging Face y GitHub Pages	Exploración de una demostración web complementaria

Aclaración técnica: los agentes configurados en OpenCode y los especialistas invocados por el programa Python no están integrados entre sí. Son dos formas de trabajo desarrolladas dentro del mismo proyecto.

---

4. Desarrollo cronológico de actividades

Etapa 1. Planeación y elección de herramientas

Actividad realizada. Comenzamos definiendo el propósito del sistema y las funciones que tendría cada agente. Decidimos trabajar con un Coordinador y dos especialistas, uno en Catastro y otro en Geodesia, porque queríamos observar cómo se distribuyen las consultas según el tema. También consideramos que el proyecto debía ejecutarse en un computador personal, sin depender necesariamente de servicios comerciales de inteligencia artificial.

Procedimiento. Revisamos las herramientas disponibles y seleccionamos OpenCode para experimentar con agentes, Ollama para administrar modelos locales y Python para desarrollar una solución que pudiéramos inspeccionar y probar directamente. La disponibilidad de memoria y capacidad de procesamiento del equipo fue un criterio importante al elegir los modelos.

Resultado y aprendizaje. Establecimos una arquitectura inicial y distinguimos dos necesidades: configurar agentes en un entorno existente y disponer de un programa propio que permitiera verificar la selección de especialistas.

Etapa 2. Instalación de Ollama y descarga de Qwen

Actividad realizada. Instalamos Ollama y descargamos modelos de la familia Qwen para realizar consultas locales.

Comandos utilizados:

```
ollama pull qwen3:4b
ollama pull qwen2.5:3b
ollama pull qwen2.5:7b
ollama list
```

Dificultad encontrada. Comprobamos que descargar un modelo no significa que OpenCode pueda utilizarlo automáticamente: es necesario definir el proveedor, la dirección de conexión y el nombre correcto del modelo.

Resultado y aprendizaje. Los modelos quedaron disponibles en Ollama para las pruebas del proyecto. También entendimos que los modelos de mayor tamaño pueden exigir más recursos y tardar más en responder.

Etapa 3. Configuración de OpenCode

Actividad realizada. Preparamos el archivo opencode.json para conectar OpenCode con Ollama como proveedor local.

Procedimiento. Indicamos la dirección del servicio http://127.0.0.1:11434/v1, registramos los modelos disponibles y asignamos modelos a los agentes. Después abrimos OpenCode desde la carpeta del proyecto para comprobar que reconociera la configuración.

```
opencode
```

Pruebas iniciales. Realizamos consultas sencillas para verificar la conexión y luego formulamos preguntas técnicas. Estas pruebas permitieron confirmar la interacción con el modelo, aunque no demostraron por sí solas una delegación automática entre todos los agentes.

Resultado y aprendizaje. Conseguimos utilizar modelos locales desde OpenCode y comprendimos mejor la relación entre la configuración del proveedor y las instrucciones de los agentes.

Etapa 4. Creación del Coordinador y los especialistas

Actividad realizada. Definimos tres archivos de instrucciones en .opencode/agents/:

```
.opencode/
└── agents/
    ├── coordinador.md
    ├── catastro.md
    └── geodesia.md
```

Procedimiento. Asignamos al Coordinador la función de identificar la disciplina de una consulta. El especialista de Catastro se enfocó en identificación predial, Zonas Homogéneas Físicas (ZHF), Zonas Homogéneas Geoeconómicas (ZHG) y avalúos. El especialista de Geodesia se orientó a MAGNA-SIRGAS, sistemas de referencia, coordenadas y proyecciones, incluido el sistema MAGNA-SIRGAS / Origen-Nacional (EPSG:9377).

Resultado y aprendizaje. Los agentes quedaron definidos con responsabilidades diferentes. Esta separación nos ayudó a comprender que una arquitectura multiagente necesita instrucciones y criterios de asignación claros, no solo varios nombres de agentes.

Etapa 5. Pruebas de coordinación y errores en OpenCode

Actividad realizada. Intentamos que el Coordinador derivara preguntas a los especialistas desde OpenCode.

Dificultad encontrada. Algunas pruebas de delegación generaron errores en la herramienta de subagentes, incluido el mensaje Invalid arguments for tool "subagent". También realizamos consultas directas a especialistas, como una pregunta sobre MAGNA-SIRGAS, que sí produjeron respuestas.

Decisión tomada. En lugar de considerar resuelta la delegación nativa, documentamos la limitación y continuamos con una implementación independiente en Python para comprobar de manera más controlada la selección y ejecución de especialistas.

Resultado y aprendizaje. Diferenciamos entre tres cosas: que el modelo responda, que un especialista responda directamente y que un Coordinador delegue automáticamente. Solo las dos primeras se observaron en las pruebas descritas de OpenCode.

Etapa 6. Implementación funcional en Python

Actividad realizada. Desarrollamos el archivo principal sistema_multiagente.py, conectado directamente con Ollama. Trabajamos hasta una versión identificada como **V5.2**.

Procedimiento. El programa recibe una pregunta, aplica reglas de clasificación para elegir Catastro, Geodesia o ambos, recupera información de documentos locales y solicita respuestas a los modelos Qwen. Cuando participan los dos especialistas, agrega una síntesis interdisciplinaria mediante una regla explícita.

Mejoras incorporadas. Añadimos un validador basado en patrones para identificar determinados errores conocidos. Cuando encuentra alguno, el programa intenta una corrección; si el problema persiste, puede bloquear la respuesta. Esta verificación no equivale a una revisión factual completa.

Resultado y aprendizaje. Obtuvimos una versión que podía ejecutarse desde la terminal y en la que era posible observar qué agentes habían sido seleccionados y qué documentos se habían consultado.

Etapa 7. Organización de documentos técnicos

Actividad realizada. Creamos carpetas separadas para documentos de Catastro y Geodesia.

Procedimiento. Incorporamos archivos TXT de apoyo y documentos PDF técnicos relacionados con ZHF, ZHG, valoración catastral, MAGNA-SIRGAS y Origen-Nacional. El programa utiliza estos materiales como contexto para elaborar las respuestas.

```
documentos/
├── catastro/
│   ├── 01_zonas_homogeneas.txt
│   ├── 01_zonas_homogeneas_IGAC.pdf
│   └── 02_identificacion_valoracion.txt
└── geodesia/
    ├── 01_magna_sirgas.txt
    ├── 02_origen_nacional.txt
    └── 02_origen_nacional_IGAC.pdf
```

Resultado y aprendizaje. Logramos incorporar referencias documentales locales. También identificamos que proporcionar documentos al modelo no elimina la necesidad de revisar críticamente las afirmaciones generadas.

Etapa 8. Pruebas y verificación de resultados

Actividad realizada. Ejecutamos verificaciones internas y tres consultas representativas del funcionamiento del programa Python.

Verificación básica:

```
python sistema_multiagente.py --test
```

El programa reportó **11 verificaciones básicas superadas**. Estas comprobaciones evaluaron aspectos operativos del código, no la veracidad de todas las respuestas.

Prueba	Agente seleccionado	Documentos consultados	Tiempo aproximado	Observación
MAGNA-SIRGAS y EPSG:9377	Geodesia	2	14,45 s	Selección correcta y respuesta generada

Prueba	Agente seleccionado	Documentos consultados	Tiempo aproximado	Observación
ZHF, ZHG y avalúos	Catastro	1	12,19 s	Respuesta generada; se detectó una afirmación incorrecta no señalada por el validador
Relación entre EPSG:9377 y ZHF/ZHG	Catastro y Geodesia	3	15,24 s	Participaron ambos especialistas y se presentó una síntesis

Análisis de las pruebas. La primera consulta permitió comprobar la selección del especialista geodésico. La segunda mostró que el sistema puede seleccionar Catastro y utilizar un PDF técnico, pero también evidenció una limitación: el modelo atribuyó a las zonas homogéneas efectos sobre tasas tributarias que no correspondían, sin que el validador lo advirtiera. En la tercera consulta se observó la activación de ambos especialistas y la generación de una síntesis, lo que confirmó el funcionamiento interdisciplinario de la implementación Python.

Resultado y aprendizaje. Las pruebas respaldan que el programa funciona operativamente en el entorno utilizado. No demuestran que todas las respuestas sean exactas ni que el sistema esté validado en otros computadores.

Etapa 9. Preparación de archivos para la entrega

Actividad realizada. Organizamos una carpeta destinada a la entrega y una copia del proyecto en el repositorio local GeoCat_IA_GitHub.

Procedimiento. Conservamos el archivo Python principal, las configuraciones de OpenCode, los documentos de consulta y los archivos necesarios para la instalación. También incorporamos requirements.txt, instalar.bat e iniciar.bat, con el propósito de facilitar la preparación y ejecución del programa en Windows.

Resultado y aprendizaje. Comprendimos que entregar un proyecto no consiste únicamente en compartir el código: también es necesario explicar sus dependencias, su estructura y las condiciones de ejecución.

Etapa 10. Exploración de una demostración web

Actividad realizada. Exploramos una presentación web del proyecto mediante GitHub Pages y un espacio de Hugging Face.

Dificultad encontrada. La disponibilidad de recursos de ejecución y las restricciones de cuota de ZeroGPU impidieron considerar esta demostración como una forma confiable de ejecutar los modelos en todo momento.

Decisión tomada. Mantuvimos la ejecución local como vía principal para las pruebas y GitHub como medio de entrega del código. La demostración web quedó como componente complementario.

Resultado y aprendizaje. Diferenciamos entre publicar una página accesible y disponer de capacidad de cómputo suficiente para responder consultas de inteligencia artificial.

Etapa 11. Publicación del repositorio en GitHub

Actividad realizada. Publicamos el proyecto en el repositorio **GeoCat-IA**, dentro de la cuenta NikFlorez3.

Procedimiento. Utilizamos Git desde PowerShell para preparar los archivos, registrar los cambios y enviarlos a la rama principal:

```
git add .
git commit -m "Publicar proyecto GeoCat IA"
git push origin main
```

Los comandos anteriores ilustran el procedimiento; los mensajes de commit concretos pueden variar. Confirmamos que la publicación se completó al observar la respuesta exitosa de Git y los archivos en GitHub.

Resultado y aprendizaje. El repositorio permitió reunir el código, los agentes, los documentos y las instrucciones de instalación en un mismo lugar para que el docente pudiera revisarlos.

Etapa 12. Corrección del README y actualización final

Actividad realizada. Revisamos el archivo README.md y detectamos que algunos títulos, negrillas y comandos no se visualizaban como esperábamos debido a caracteres de escape y problemas de formato.

Procedimiento. Corregimos el texto Markdown, reorganizamos los apartados de instalación, arquitectura, pruebas y limitaciones, y guardamos el archivo actualizado. Después ejecutamos:

```
git add README.md
git commit -m "Mejorar documentacion y presentacion de GeoCat IA"
git push origin main
```

Resultado comprobado. El envío a GitHub se completó sin errores y la rama main avanzó del commit e94f559 al commit 3194647.

Resultado y aprendizaje. Aprendimos que la documentación también forma parte del proyecto y que un README bien estructurado facilita su instalación, revisión y presentación académica.

---

5. Dificultades principales y soluciones aplicadas

Dificultad	Acción realizada	Estado al finalizar
Conectar OpenCode con modelos locales	Configuración de Ollama en opencode.json	Conexión y consultas directas probadas
Delegación automática de subagentes nativos	Revisión de configuración y pruebas	No completamente validada
Control de errores técnicos en respuestas	Validador por patrones e intento de corrección	Implementado, con limitaciones detectadas
Disponibilidad de demostración web	Exploración de GitHub Pages y Hugging Face	Complementaria; ejecución no garantizada
Visualización incorrecta del README	Corrección de Markdown y nuevo push	Actualización publicada

6. Resultados generales

Al finalizar el trabajo contamos con una configuración de tres agentes en OpenCode y una implementación Python capaz de clasificar consultas de Catastro y Geodesia, seleccionar especialistas, recuperar documentos locales y producir respuestas. También realizamos una prueba interdisciplinaria, superamos 11 verificaciones básicas del programa y publicamos los archivos del proyecto en GitHub.

Estos resultados corresponden a las pruebas locales descritas. No implican que la delegación nativa de OpenCode esté resuelta ni que todas las respuestas de los modelos sean correctas.

7. Reflexión y conclusiones

El desarrollo de GeoCat IA nos permitió aplicar herramientas que inicialmente conocíamos de manera separada. Aprendimos a configurar modelos locales, crear instrucciones para agentes, estructurar un programa Python y registrar cambios con Git. También entendimos la importancia de documentar los errores en lugar de ocultarlos: las dificultades de delegación en OpenCode y la afirmación incorrecta que apareció en una respuesta de Catastro mostraron que una prueba exitosa de ejecución no equivale a una validación técnica completa.

Consideramos que el principal resultado fue construir una base funcional y documentada para explorar la colaboración entre especialidades catastrales y geodésicas. Como trabajo futuro, sería conveniente mejorar la verificación de afirmaciones técnicas, probar la instalación en otro computador y continuar evaluando la delegación nativa entre agentes de OpenCode.

8. Referencias y recursos consultados

- OpenCode. Sitio oficial: https://opencode.ai
- Ollama. Sitio oficial: https://ollama.com
- Python Software Foundation. https://www.python.org
- Instituto Geográfico Agustín Codazzi (IGAC). https://www.igac.gov.co
- GitHub. https://github.com
- Hugging Face. https://huggingface.co
- **Repositorio del proyecto:** https://github.com/NikFlorez3/GeoCat-IA

Bitácora académica elaborada por Nikole Flórez y Diana Avendaño — 2026.